

Algorithms for Data Science

Exercise 3

See the cheatsheet for valuable remarks!

Task 3.1: Divide and Conquer

Write a recurrence for the running time of the following algorithms in dependence of n . Write it in the form $T(n) = aT(n/b) + f(n)$.

Then solve the recurrences!

(a)

```
StrangeAlgo1(n)
  if n > 1 then
    i = 1
    while i ≤ n do
      i ++
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
  return 1;
```

(b) What is the running time of the algorithm above if we change the while loop condition from $i \leq n$ to $i \leq n \cdot \log_2 n$?

(c) And what if we change the while loop condition to $i \leq n^2 / \log_2 n$ instead? Can we apply the Master Theorem?

(d)

```
StrangeAlgo2(n)
  if n > 1 then
    for i = 1 to n do
      for j = 1 to n do
        k = j
        while k ≤ n do
          k = k + 1
      for i = 1 to 8 do
        StrangeAlgo2(n/2)
  return 1;
```

(e)

```

StrangeAlgo3(a[1..n])
  if n < 2 then
    return a[1]
  else
    ℓ = ⌊n/5⌋
    a' = a[1..ℓ]
    b' = a[2..ℓ + 1]
    p = StrangeAlgo3(a')
    a'[1] = p - 1
    return min(StrangeAlgo3(a'), StrangeAlgo3(b'))

```

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 3.2: Master Theorem

Solve the following recurrences using the master theorem. *First, determine $f(n)$ and the values of a and b . Secondly, compute c . Finally, conclude which case of the master theorem holds (by determining an appropriate c' if Case A holds) and write the solution.*

In the following, you can assume that $f(n)$ satisfies the regularity condition of Case C provided that the first condition of Case C holds.

(a)

$$T(n) = 5T(n/5) + \Theta(\log n)$$

(b)

$$T(n) = 3T(n/9) + \sqrt{n} + n^{1/4}$$

(c)

$$T(n) = 10^{20}T(\lceil n/100 \rceil) + 2^{\log(n^9)} \cdot \log n$$

(d)

$$T(n) = n + T(\lfloor n/9 \rfloor)$$

(e)

$$T(n) = 14n + \sqrt{10000} - 4n + T(\lceil n/10 \rceil) - 10n$$

Task 3.3: Simplifications of Recurrences

Rewrite the following recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ (where a and b are constants) simplifying $f(n)$ as far as possible. You don't have to solve the recurrences.

(a)

$$T(n) = 2T(n/3) + 42n^2 + 100n + T(n/3)$$

(b)

$$T(n) = 13 + 20 \log n + T(n/7) + 2(\sqrt{n} + T(n/7))$$

(c)

$$T(n) = \log(n^3) + n^{\log_4 1} \cdot T(n/10) + (\log n)^2 + \log(1024) + \log(\log n)$$

Task 3.4: Divide and Conquer++

For each of the following algorithms, write a recurrence for the running time and a recurrence for the *memory complexity*. Write the recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ simplifying $f(n)$ as far as possible. You don't have to solve the recurrences. (For the memory complexity, use M instead of T .)

(a)

```
StrangeAlgo4(a[1..n])
  if n < 2 then
    return a[1]
  else
    ℓ = ⌊n/7⌋
    a' = a[1..ℓ]
    b' = a[2..ℓ + 1]
    i = 1
    while i ≤ ℓ do
      i ++
    k = 1
    while k < 7 do
      p = StrangeAlgo4(a')
      a'[1] = p - 1
      k ++
    i = 1
    while i ≤ n do
      j = 1
      while j ≤ n do
        j ++
        b'[1] = min(a[i], a[j])
      i ++
    return max(StrangeAlgo4(a'), StrangeAlgo4(b'))
```

(b)

Input: An array $a[1..n]$ whose all elements are pairwise different and whose length n is a power of 2, i.e., $n = 2^d$ for some integer d .

Output: The array a is sorted increasingly.

```
SlowQuickSort( $a[1..n]$ )
  if  $n < 2$  then
    return  $a$ 
  pivot = 1
   $a1[1..n][1..n]$  new empty two-dimensional array
   $a2[1..n][1..n]$  new empty two-dimensional array
   $p1 = 1$ 
   $p2 = 1$ 
  while pivot  $\leq n$  do
    for  $i = 1$  to  $n$  do
      if  $a[i] < a[pivot]$  then
         $a1[pivot][p1] = a[i]$ 
         $p1 = p1 + 1$ 
      else if  $a[i] \geq a[pivot]$  then
         $a2[pivot][p2] = a[i]$ 
         $p2 = p2 + 1$ 
    if  $p1 == n/2 + 1$  and  $p2 == n/2 + 1$  then
      break // Leaves the while loop immediately
    pivot ++
     $p1 = 1$ 
     $p2 = 1$ 
   $b1[1..p1-1]$  new empty array
   $b2[1..p2-1]$  new empty array
  for  $i = 1$  to  $n/2$  do
     $b1[i] = a1[pivot][i]$ 
     $b2[i] = a2[pivot][i]$ 
   $b1 = \text{SlowQuickSort}(b1)$ 
   $b2 = \text{SlowQuickSort}(b2)$ 
  for  $i = 1$  to  $n/2$  do
     $a[i] = b1[i]$ 
     $a[i + n/2] = b2[i]$ 
  return  $a$ 
```

(c) What property has the element $a[pivot]$ immediately after the while loop is left?

(d) What happens, if we run $\text{SlowQuickSort}(a)$ with an array a where all the elements are the same?

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 3.5: Here and There

You are given two arrays $here[1..n]$ and $there[1..n]$ containing numbers from $\{1, 2, 3\}$. There are n smileys numbered from 1 to n . For $1 \leq i \leq n$, $here[i]$ is the initial position of smiley i and $there[i]$ is the position it wishes to be. There are two rules:

1. All smileys on the same position are stacked over each other such that smileys with larger number are below smileys with smaller number.
2. A single step consists of taking one smiley from the top of one position and placing it on the top of some other position.

Write a function `fulfill_wishes` that writes how to fulfill the wishes of all the smileys (at the same time and without violating the rules at any point)! The output of your function should be a sequence of move operations of the form “Take the top-most smiley from position i and place it on top of position j ”.